

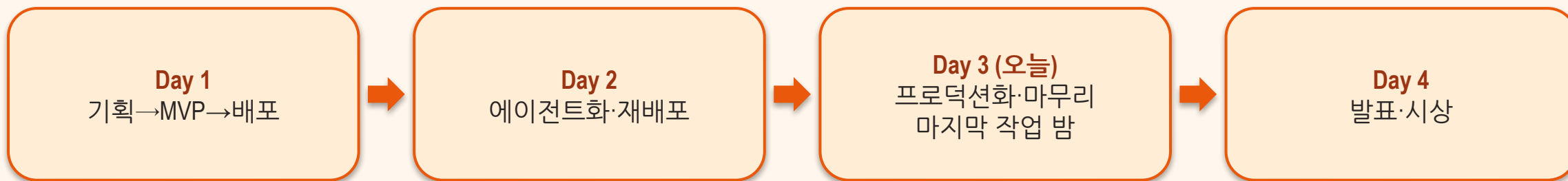
23차시 · 팀 프로젝트 (매일 저녁 19:00~21:00)

팀 프로젝트 Day 3 — 프로덕션화 · 마무리

믿고 쓸 수 있는 상태로 — 그리고 발표까지 갈 모든 걸 오늘 밤 마무리

오늘이 발표 전 마지막 작업 밤

▶ 오늘 안 끝내면 내일은 리허설뿐 — 코드·배포·영상은 오늘이 마감



오늘의 리듬 — 원칙 3

① 믿고 쓸 상태

운영 품질을 하나라도 실제로 붙인다

② Langfuse는 로컬만

로컬 = 측정·캡처(발표 증거) · 배포본 = 가드레일·로그(코드)

③ 오늘이 마지막 밤

머지 · 최종 배포 · 시연 영상 · 발표 준비까지 오늘 마무리

! 전제 — Langfuse는 로컬에서만 · 운영을 두 갈래로

▶ 배포된 실서버엔 붙일 Langfuse가 없다

로컬에서
(측정 → 캡처)

트레이싱·평가·프롬프트를 Langfuse로 측정 → 스크린샷·수치로 캡처 = 발표 '운영·품질' 증거

배포본에
(코드로만)

실서버엔 Langfuse 없음 → 가드레일 + 간단 로깅(console / Vercel 함수 로그)만 올라간다

한 줄: Langfuse 화면 = 발표용 증거(로컬) · 배포본 운영 = 가드레일·로그(코드)

SECTION 01

오늘 밤의 미션

DoD @ 운영·품질 — 아래 중 최소 2개 (로컬 측정 + 캡처)

▶ 앞 3개는 로컬 캡처 · 가드레일만 배포본에 실제로 올라감

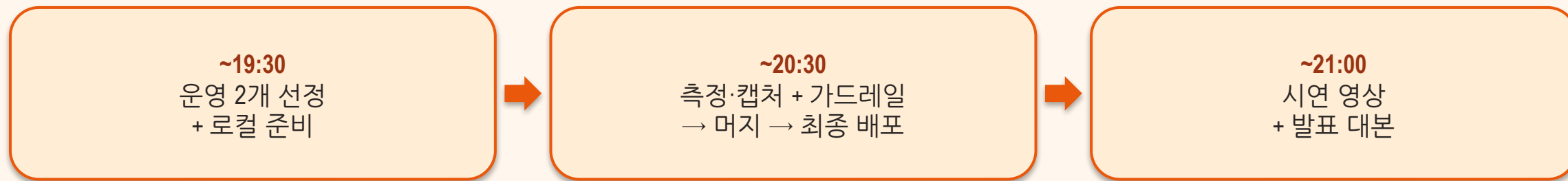
- □ 평가셋으로 핵심 기능 통과/실패 측정 (로컬 Langfuse) → 수치·화면 캡처
- □ 트레이싱(요청·비용·지연) 확인 (로컬 Langfuse) → 캡처
- □ 프롬프트 버전 관리로 개선 전/후 비교 (로컬 Langfuse) → 캡처
- □ 배포본에 입출력 가드레일 1개 (코드 — 실서버에 올라감)
- □ 로그 기반 개선 1사이클(전/후 비교)

DoD ⑥ 마무리 — 4개 전부 필수

▶ 미루면 발표 때 없다

- □ 개발 코드 전부 `main`에 머지 — 흩어진 feature 브랜치 정리·충돌 해결
- □ 최종 버전 실서버 배포 — 어제와 같은 URL · 전원 재접속 확인
- □ 시연(백업) 영상 녹화 — 핵심 flow 60~90초 (라이브 불안 시 대체)
- □ 발표 준비 착수 — 데모 대본 초안 + 발표 역할 분담(발표 시간은 10~15분)

예시 타임라인 — 마일스톤 3개



SECTION 02

STEP 1 · ~19:30 — 운영 항목 선정

약한 곳 찾기 — 선정 질문 4

▶ 약점 없는 팀은 없다

1

데모에서 가장 자주 틀리는 답은?

→ 평가셋 / 프롬프트 개선 후보

2

비용·지연이 튀는 요청은?

→ 트레이싱으로 측정

3

위험한 출력은?(환각·욕설·개인정보)

→ 가드레일 후보

4

1시간 안에 측정·개선 가능한 2개는?

답 못하면 범위를 줄인다

~19:30

운영 4트랙 — 하나씩 골라 붙이기

▶ 앞 3개는 로컬 Langfuse · 가드레일은 코드로 배포본에

평가셋
(15·20차시)

통과율 측정 — 로컬 캡처

트레이싱
(17·18차시)

비용·지연 확인 — 로컬 캡처

프롬프트 버전
(19차시)

전/후 비교 — 로컬 캡처

가드레일
(21차시)

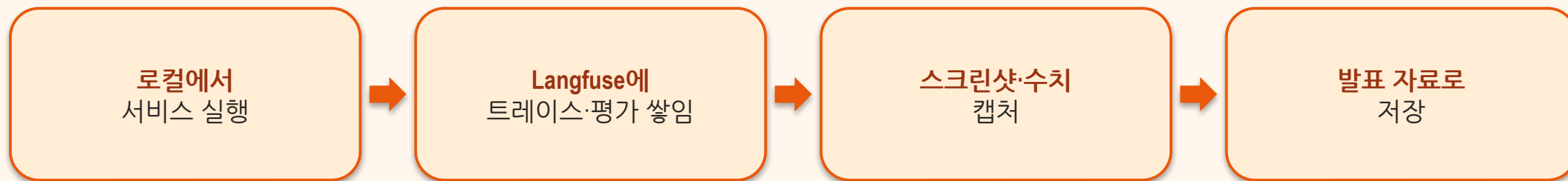
검증·폴백 — 배포됨

SECTION 03

STEP 2 · ~20:30 — 측정·캡처 → 가드레일 → 머지 → 배포

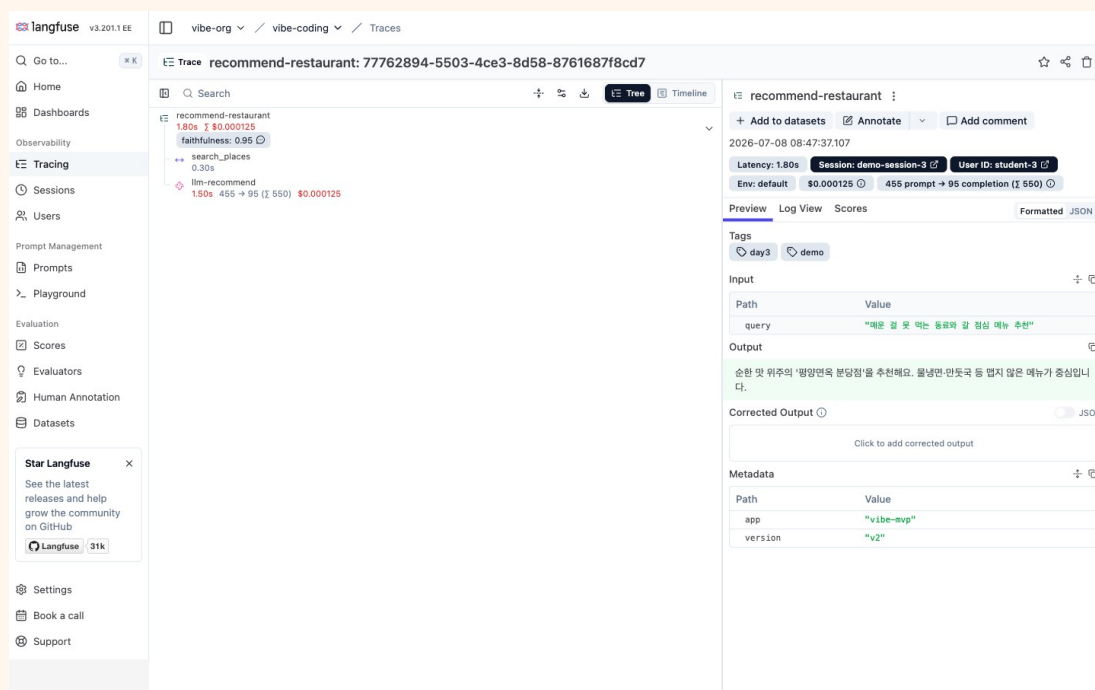
로컬에서 측정 → 캡처 흐름

▶ 캡처가 곧 산출물 — 화면만 보고 넘기면 발표 때 증거가 없다

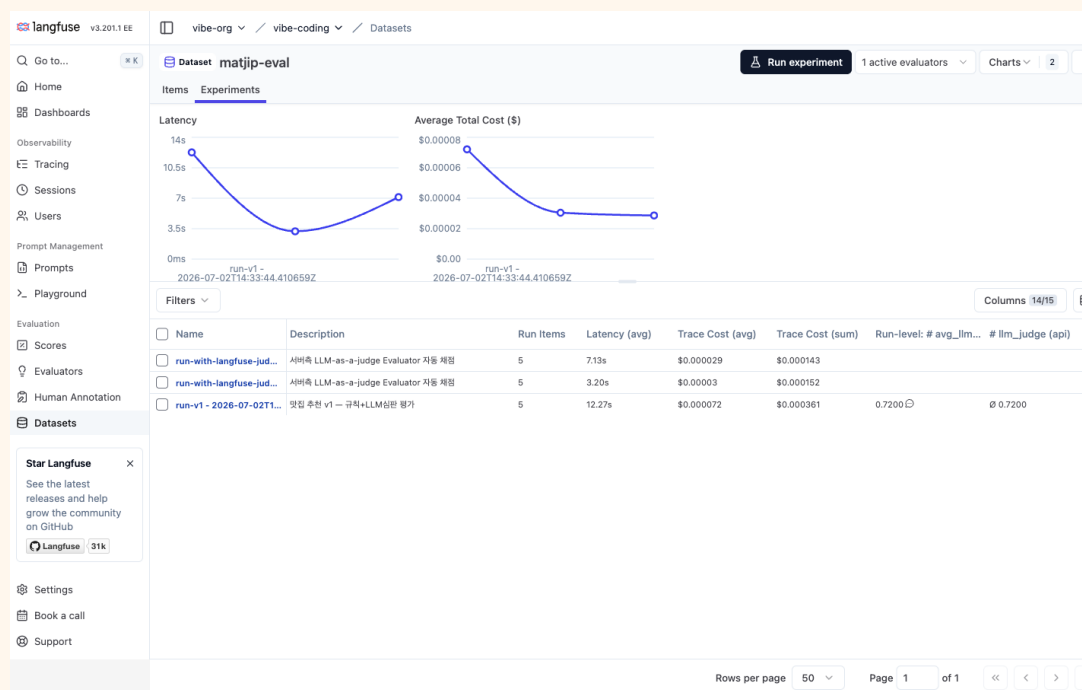


~20:30

로컬 Langfuse 화면 — 트레이스 · 실험 (이런 1~2장이 증거)



① 트레이스 상세 — 요청·비용·지연



② 실험 비교 — 프롬프트/모델 전·후

가드레일 1개 — 배포본에 코드로 (Langfuse 불필요)

▶ 가드레일은 Langfuse 없이도 도는 코드 → 배포본에 넣는 유일한 운영

```
// 서버 함수 안 — 입력·출력 검증 (실서버에 실제로 올라감)
function guard(input, output) {
  if (!input?.trim())      return '질문을 입력해 주세요';    // 입력 검증
  if (isRisky(input))      return '도와드릴 수 없는 요청이에요'; // 위험 입력 차단
  if (!hasEvidence(output)) return '잘 모르겠어요';          // 근거 없으면 풀백
  return output;
}
```

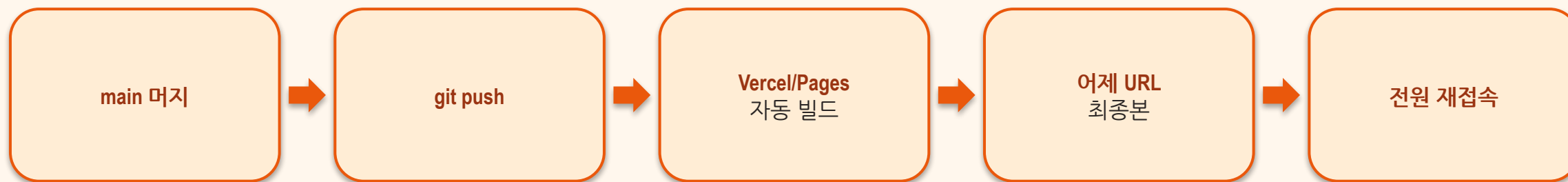
개발 코드 머지 — 흠어진 브랜치를 main 으로

▶ 배포는 main 기준 — 오늘 밤 다 모은다

```
# 담당자별로 feature 브랜치를 main 에  
git checkout main && git pull origin main  
git merge feature/agent      # 충돌 나면 파일 담당자끼리 해결  
git merge feature/ui  
npm run dev                  # 머지 후 로컬 전체 동작 한 번 확인  
git push origin main         # → 자동 재배포 트리거
```

최종 실서버 배포 — 같은 URL

▶ 이게 발표에서 보여줄 최종 URL



배포 env엔 **OpenAI** 키만 — **Langfuse** 키는 넣지 말 것(실서버엔 Langfuse 없음).
실패 시 롤백: **Vercel Deployments** 탭 → 이전 배포 **Promote** — 어제 URL로 즉시 복귀.

배포 후 검증 3분

▶ '로컬에선 됐는데'를 오늘 밤 졸업 — 배포본 기준으로 확인

1

핵심 flow 1개 배포 URL에서 직접
브라우저에서 실제로 한 번 끝까지 돌려보기

2

가드레일 동작 확인
일부러 빈/이상 입력 넣어 폴백이 뜨나 확인

3

함수 로그(Vercel) 훑기
500·undefined 없나 — 있으면 env 미등록/재배포 확인

SECTION 04

STEP 3 · ~21:00 — 시연 영상 · 발표 준비 · 제출

시연(백업) 영상 — 60~90초

▶ 라이브가 최선이지만 백업 영상은 보험 — 오늘 배포된 화면으로 지금 찍는다

1

무엇을

핵심 flow 1개 — 문제 → 해결 → Wow 순간

2

어떻게

화면 녹화(QuickTime/OBS) · 배포 URL에서 · 자막/내레이션 짧게

3

왜

발표 라이브 불안 시 대체

발표 준비 착수 — 대본 4요소 + 역할

① AI 서비스 데모

배포 URL에서 핵심 flow 라이브 (또는 백업 영상)

② MVP 전략

무슨 문제를 왜 이렇게 좁혀 풀었나

③ Agent 구조

입력→계획→도구/RAG→(반성)→응답 · '왜 이 방식'

④ 테스트·운영 전략

오늘 캡처한 **Langfuse** 화면 + 가드레일을 근거로

5명 역할 분담: 발표 · 데모 조작 · Q&A 대비까지 지금 정한다.

목표 산출물

- ☐ 운영 2개 캡처 (로컬 Langfuse / 가드레일)
- ☐ 머지 커밋 해시
- ☐ 최종 배포 URL — 어제와 동일
- ☐ 시연 영상 링크
- ☐ 발표 대본 초안

SECTION 05

마무리

클로징 — 오늘 밤의 완성본

- 운영 품질(로컬 측정·캡처) + 가드레일(배포) + 전 코드 머지 → 최종 배포 + 시연 영상 + 발표 대본
- 3일간 자란 서비스가 오늘 밤 완성본 — 내일 이 URL로 발표